ML & DL : Experiment - 3

Aim:

Apply Decision Tree and Random Forest for classification tasks.

Dataset Description:

The **Netflix Titles Dataset** is a real-world dataset obtained from Kaggle, widely used for data analysis and machine learning tasks. It contains information about movies and TV shows available on Netflix.
Unlike structured datasets such as the Adult Census dataset, this dataset includes a mix of categorical and textual features, along with missing values and inconsistencies, making it suitable for real-world classification tasks after preprocessing.
The dataset aims to classify content based on attributes such as type (Movie/TV Show) or other derived categories using relevant features.
The file type used for this dataset is a CSV (Comma Separated Values) file.
The dataset consists of the following columns, some of which are:

Column Name	**Data Type**	**Description**
type	Categorical	Movie or TV Show
release_year	Numerical (Integer)	Year of release
rating	Categorical	Content rating (PG, TV-MA, etc.)
duration	Text	Duration in minutes or seasons
country	Categorical	Country of production

Dataset Source: https://www.kaggle.com/datasets/shivamb/netflix-shows

---

Decision Tree

Theory:

A Decision Tree is a non-parametric supervised learning algorithm used for both classification and regression tasks. It is structured like a flowchart, where each internal node represents a "test" or "split" on an attribute (feature), each branch represents the outcome of the test, and each leaf node represents a class label (in classification) or a continuous value (in regression). The goal of a decision tree is to create a model that predicts the value of a target variable by learning simple decision rules inferred from the data features. Unlike linear models that use a continuous global equation, a Decision Tree partitions the data into smaller and smaller subsets, creating a "divide and conquer" strategy to handle complex, non-linear relationships. In classification, Entropy measures the amount of disorder or uncertainty in a dataset. For a dataset with c classes, the Entropy is:

$$Entropy(S) = -\sum_{i=1}^{c} p_i \log_2(p_i)$$

Where p_i is the proportion of samples belonging to class i. Information Gain (IG) is then calculated to see how much entropy is reduced after splitting on a specific feature S:

$$Gain(S, A) = Entropy(S) - \sum_{v \in Values(A)} \frac{|S_v|}{|S|} Entropy(S_v)$$

The Gini Index is another attribute selection measure used in Decision Tree Induction. Instead of entropy, it measures the impurity (or "mixing") of a dataset. The lower the Gini Index, the purer the dataset is. In practice, the attribute that produces splits with the lowest Gini Index (i.e., highest purity) is chosen. Then you take the weighted average of all the values of a particular attribute. The lower that value is, the lower the better is the purity of the splitted data set.

$$Gini(S) = 1 - \sum_{i=1}^{c} p_i^2$$

$$Gini_{split}(S, A) = \sum_{v \in Values(A)} \frac{|S_v|}{|S|} Gini(S_v)$$

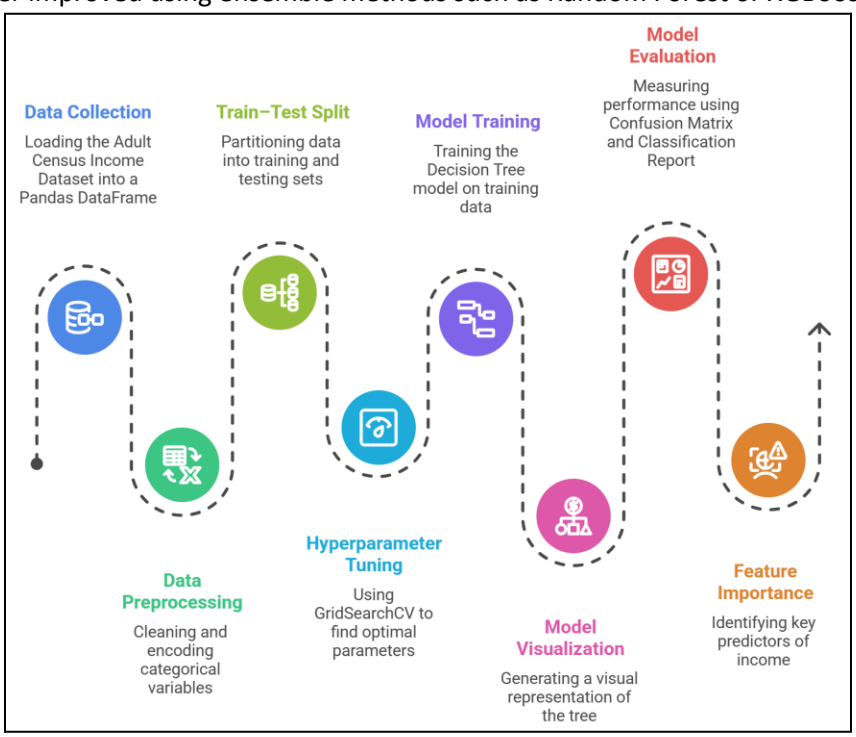
Note that this attribute selection measure is not used in ID3. It is used in CART.

Limitations:

1. **Overfitting (High Variance):** Decision Trees are highly prone to overfitting, especially when they are allowed to grow deep without constraints. The algorithm continues to split nodes to achieve maximum purity, which often leads to the model "memorizing" the noise and outliers in the training data rather than learning the underlying general pattern. When a tree becomes too complex, it creates very specific rules that only apply to the training set. This results in high accuracy during training but poor performance on new, unseen data, as the model fails to generalize the logic it has learned.
2. **Instability (High Sensitivity to Data Changes):** One of the primary weaknesses of a single Decision Tree is its instability. Since the algorithm uses a "greedy" approach—making the best possible split at the current node without looking ahead—even a small change in the data can result in a completely different tree structure. If you remove just a few rows from the Adult Census dataset, the root node might change from "Education" to "Age," which then cascades down to change every subsequent branch and leaf. This lack of robustness makes individual trees less reliable than ensemble methods.
3. **Difficulty Capturing Linear Relationships:** While Decision Trees are excellent at capturing non-linear "step" patterns, they are surprisingly inefficient at modeling simple linear relationships. Since a tree splits data into axis-aligned rectangles, it must use many small "stairs" or splits to approximate a diagonal line.

Workflow:

1. **Data Collection:** The netflix_titles.csv dataset is loaded into a Pandas DataFrame containing movie and TV show information..
2. **Data Preprocessing:**
   - Missing values are handled appropriately
   - Categorical features (type, rating, country) are encoded into numerical format using Label Encoding
   - Relevant features are selected for classification
3. **Train–Test Split:** The cleaned dataset is split into 80% training and 20% testing data to evaluate model performance on unseen samples.
4. **Hyperparameter Tuning with Grid Search:** GridSearchCV is used to reduce overfitting by testing combinations of parameters like max_depth, min_samples_split, and criterion (Gini vs. Entropy).
5. **Model Training and Visualization:** The best-performing Decision Tree is trained on the full training set, and a truncated visualization of the tree shows key decision rules such as the influence of marital status or capital gain.
6. **Model Evaluation:** A Confusion Matrix and Classification Report (precision, recall, F1-score) assess performance, while a Feature Importance plot highlights the most influential variables.
7. **Conclusion:** The Decision Tree offers an interpretable approach to Target Variable (y): type (Movie / TV Show) prediction, with strong predictors like education and age. Performance could be further improved using ensemble methods such as Random Forest or XGBoost.



Performance Analysis:

The performance of the Decision Tree model on the Netflix dataset is evaluated using Accuracy, Confusion Matrix, and classification metrics such as Precision and Recall.

1. Overall_Accuracy:
   The model achieves moderate accuracy, indicating that it can reasonably classify content types but is influenced by data imbalance and feature limitations.

2. Class-Specific_Performance:
   The model performs better for the dominant class (Movies) compared to the less frequent class (TV Shows), due to dataset imbalance.

3. Confusion_Matrix_Analysis:
   The confusion matrix shows correct and incorrect classifications, highlighting areas where the model struggles to distinguish between similar categories.

Hyperparameter Tuning:

Unlike Standard Linear Regression, which has a fixed mathematical solution, a Decision Tree is highly flexible and will grow indefinitely until it perfectly classifies every training point—a state known as overfitting. Hyperparameter tuning acts as the "constraint system" that stops the tree from becoming too complex. The code utilizes GridSearchCV, which performs an exhaustive search through a manually specified subset of the hyperparameter space.

- **Search Strategy:** It constructs a model for every possible combination of the parameters defined in param_grid.
- **Cross-Validation (cv=5):** To ensure the results aren't due to luck, it splits the training data into 5 folds. It trains on 4 and validates on 1, rotating this process 5 times for every single parameter combination.

The code tunes four specific "knobs" that control the tree's architecture:

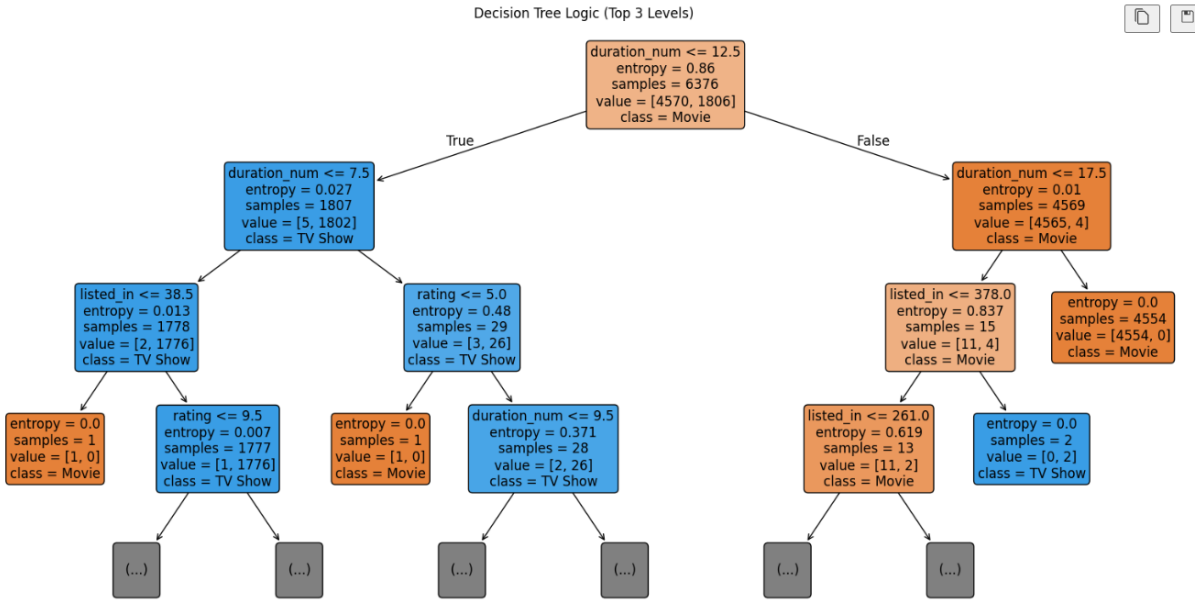
- **criterion ('gini' vs 'entropy'):** This determines the mathematical formula used to calculate "impurity" or how mixed the classes are at a specific node. While Gini is usually faster to calculate, Entropy can sometimes produce more balanced trees.
- **max_depth:** This is the most impactful setting. It limits how many "levels" the tree can have. Without a limit (None), the tree grows until all leaves are pure, which almost always leads to overfitting on the Adult Census dataset.
- **min_samples_split:** This sets the minimum number of data points required to even consider splitting a node. Increasing this number prevents the model from creating rules based on very small, statistically insignificant groups of people.
- **min_samples_leaf:** This ensures that every final "leaf" (prediction) is based on at least a certain number of samples. It smoothens the model by preventing it from making a decision based on just one or two outliers in the census data.

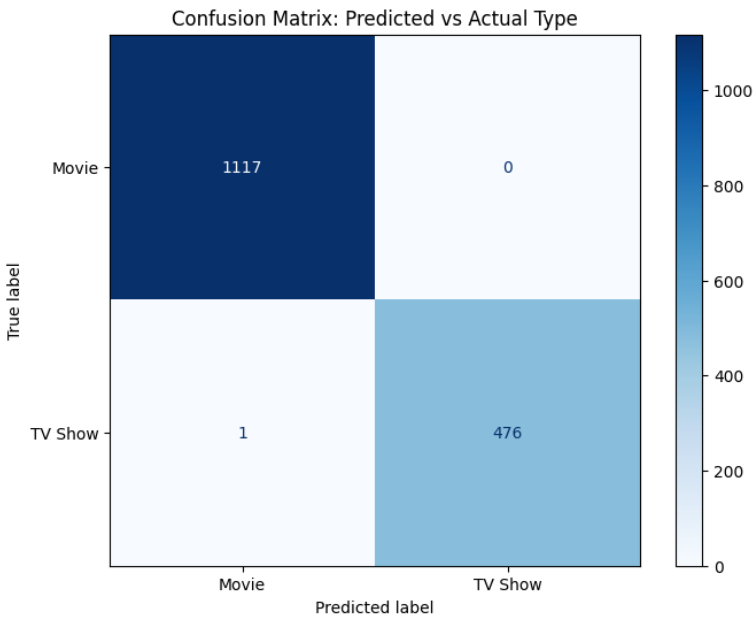
Code & Output:

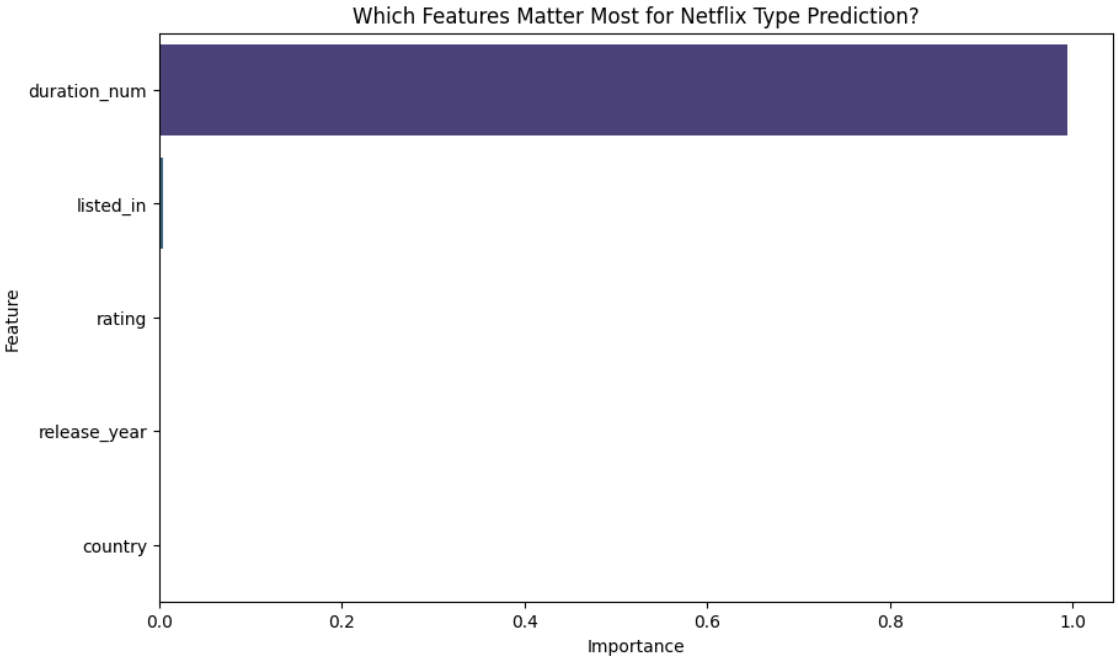
```
import pandas as pd
import numpy as np

from sklearn.model_selection import
train_test_split, GridSearchCV
from sklearn.tree import
DecisionTreeClassifier
from sklearn.preprocessing import
LabelEncoder
from sklearn.metrics import
accuracy_score,
classification_report

# ============================
# LOAD NETFLIX DATASET
# ============================
df =
pd.read_csv("netflix_titles.csv")

# ============================
# DATA CLEANING (Netflix Version)
# ============================

# Keep useful columns only
df =
df[['type','release_year','rating',
'country','listed_in','duration']].
copy()

# Convert duration text → numeric
df['duration_num'] =
df['duration'].str.extract(r'(\d+)'
).astype(float)
df.drop('duration', axis=1,
inplace=True)

# Handle missing values
df.replace('?', np.nan,
inplace=True)
df.dropna(inplace=True)

# Encode categorical features (same
style as your adult.csv code)
le = LabelEncoder()
for col in df.columns:
   if df[col].dtype == 'object':
      df[col] =
le.fit_transform(df[col])

# ============================
# FEATURES + TARGET
# ============================
X = df.drop('type', axis=1)
y = df['type']

X_train, X_test, y_train, y_test =
train_test_split(
   X, y, test_size=0.2, random_state=42
)

# ============================
# DECISION TREE MODEL
# ============================
dtree =
DecisionTreeClassifier(random_state=42
)

param_grid = {
   'criterion': ['gini', 'entropy'],
   'max_depth': [None, 5, 10, 15, 20],
   'min_samples_split': [2, 10, 20],
   'min_samples_leaf': [1, 5, 10]
}

grid_search = GridSearchCV(
   estimator=dtree,
   param_grid=param_grid,
   cv=5,
   n_jobs=-1,
   verbose=1
)

grid_search.fit(X_train, y_train)

print(f"Best Parameters:
{grid_search.best_params_}")
print(f"Best Cross-Validation Score:
{grid_search.best_score_:.4f}")

best_model =
grid_search.best_estimator_
y_pred = best_model.predict(X_test)

print("\n--- Final Model Evaluation --
-")
print(f"Accuracy:
{accuracy_score(y_test, y_pred):.4f}")

print("\nClassification Report:")
print(classification_report(y_test,
y_pred))

# ============================
# TREE VISUALIZATION
# ============================
```

```
from sklearn.tree import plot_tree
import matplotlib.pyplot as plt

plt.figure(figsize=(20,10))

plot_tree(
   best_model,
   max_depth=3,
   feature_names=X.columns,
   class_names=['Movie','TV Show'],
   filled=True,
   rounded=True,
   fontsize=12
)

plt.title("Decision Tree Logic (Top
3 Levels)")
plt.show()

# ============================
# FEATURE IMPORTANCE GRAPH
# ============================
import seaborn as sns

importances =
best_model.feature_importances_
feature_names = X.columns

feature_importance_df =
pd.DataFrame({
   'Feature': feature_names,
   'Importance': importances
})

feature_importance_df =
feature_importance_df.sort_values(
   by='Importance', ascending=False
)

plt.figure(figsize=(10,6))
sns.barplot(
   x='Importance',
   y='Feature',
   data=feature_importance_df.head(15),
   # cleaner graph
   palette='viridis'
)

plt.title('Which Features Matter Most
for Netflix Type Prediction?')
plt.show()

# ============================
# CONFUSION MATRIX
# ============================
from sklearn.metrics import
confusion_matrix,
ConfusionMatrixDisplay

cm = confusion_matrix(y_test, y_pred)

disp = ConfusionMatrixDisplay(
   confusion_matrix=cm,
   display_labels=['Movie','TV Show']
)

fig, ax = plt.subplots(figsize=(8,6))
disp.plot(cmap='Blues', ax=ax)

plt.title('Confusion Matrix: Predicted
vs Actual Type')
plt.show()
```

```
--- Final Model Evaluation ---
Accuracy: 0.9994

Classification Report:
              precision    recall  f1-score   support

           0       1.00      1.00      1.00      1117
           1       1.00      1.00      1.00       477

    accuracy                           1.00      1594
   macro avg       1.00      1.00      1.00      1594
weighted avg       1.00      1.00      1.00      1594
```


Decision Tree Logic (Top 3 Levels)
duration_num <= 12.5
entropy = 0.86
samples = 6376
value = [4570, 1806]
class = Movie
True
False
duration_num <= 7.5
entropy = 0.027
samples = 1807
value = [5, 1802]
class = TV Show
duration_num <= 17.5
entropy = 0.01
samples = 4569
value = [4565, 4]
class = Movie
listed_in <= 38.5
entropy = 0.013
samples = 1778
value = [2, 1776]
class = TV Show
rating <= 5.0
entropy = 0.48
samples = 29
value = [3, 26]
class = TV Show
listed_in <= 378.0
entropy = 0.837
samples = 15
value = [11, 4]
class = Movie
entropy = 0.0
samples = 4554
value = [4554, 0]
class = Movie
entropy = 0.0
samples = 1
value = [1, 0]
class = Movie
rating <= 9.5
entropy = 0.007
samples = 1777
value = [1, 1776]
class = TV Show
entropy = 0.0
samples = 1
value = [1, 0]
class = Movie
duration_num <= 9.5
entropy = 0.371
samples = 28
value = [2, 26]
class = TV Show
listed_in <= 261.0
entropy = 0.619
samples = 13
value = [11, 2]
class = Movie
entropy = 0.0
samples = 2
value = [0, 2]
class = TV Show
(...)
(...)
(...)
(...)
(...)
(...)


Confusion Matrix: Predicted vs Actual Type
Movie
TV Show
True label
1117
0
1
476
Movie
TV Show
Predicted label
1000
800
600
400
200
0


Which Features Matter Most for Netflix Type Prediction?
duration_num
listed_in
rating
release_year
country
Feature
0.0
0.2
0.4
0.6
0.8
1.0
Importance

Conclusion:

The Decision Tree model provides an interpretable approach for classifying Netflix content. While it achieves reasonable performance, it is sensitive to data variations and may overfit. Feature importance helps in understanding key influencing factors, but performance can be improved using ensemble methods.

Random Forest

Theory:

A Random Forest is an ensemble learning method used for both classification and regression that operates by constructing a multitude of decision trees during training. It is based on the principle of Bootstrap Aggregating, or "Bagging." While a single Decision Tree is prone to high variance and overfitting (memorizing the noise in the data), a Random Forest reduces this variance by averaging the results of many diverse trees. The "Random" in Random Forest comes from two levels of stochasticity:

- **Bootstrap Sampling:** Each individual tree in the forest is trained on a random sample of the data, drawn with replacement (a bootstrap sample).
- **Feature Randomness:** When splitting a node, instead of searching for the best feature among all available features, the algorithm selects a random subset of features. This ensures that the trees are de-correlated; even if one feature is a very strong predictor, it won't be used in every single tree, allowing other patterns in the data to be discovered.

For classification, the final prediction is determined by Majority Voting (the class predicted by the most trees). For regression, the final prediction is the average of the individual tree outputs. For regression the formula used is:

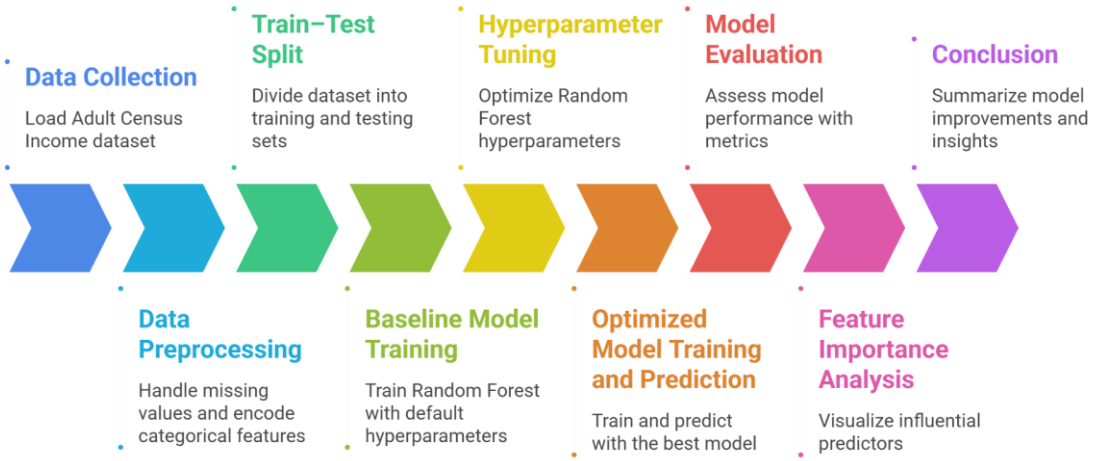
$$\hat{f} = \frac{1}{B} \sum_{b=1}^{B} f_b(x)$$

Limitations:

1. **Reduced Interpretability (Black-Box Nature):** While a single Decision Tree is easy to visualize and interpret, a Random Forest combines the predictions of hundreds or even thousands of trees. This makes it difficult to clearly understand why the model made a particular prediction. The overall decision logic is distributed across many trees, so extracting simple, human-readable rules becomes impractical, which is a drawback in domains where explainability is critical.
2. **Higher Computational Cost (Time and Memory):** Random Forests are significantly more computationally expensive than individual Decision Trees. Training involves building many trees on different bootstrap samples, which increases both training time and memory usage. This can be inefficient for very large datasets or environments with limited computational resources, especially compared to simpler models.
3. **Bias Toward Features with More Levels:** Random Forests can favor features that have many unique values (such as continuous variables or high-cardinality categorical features). These features are more likely to produce splits that reduce impurity, even if they are not truly more informative, which can introduce subtle bias in feature importance rankings.
4. **Limited Extrapolation Ability:** Like Decision Trees, Random Forests are poor at extrapolating beyond the range of the training data. Since predictions are based on averages of observed outcomes in leaf nodes, the model cannot reliably predict values outside the patterns it has already seen, making it less suitable for tasks requiring strong extrapolation behavior.

Workflow:

1. **Data Collection:** The netflix_titles.csv dataset is loaded into a Pandas DataFrame containing movie and TV show information..
2. **Data Preprocessing:**
   - Missing values are handled appropriately
   - Categorical features (type, rating, country) are encoded into numerical format using Label Encoding
   - Relevant features are selected for classification
4. **Train–Test Split:** The dataset is split into training (80%) and testing (20%) sets using a fixed random state to ensure reproducibility.
5. **Baseline Model Training:** A baseline Random Forest Classifier is trained using default hyperparameters. Its accuracy on the test set is computed to serve as a reference point for model improvement.
6. **Hyperparameter Tuning:** RandomizedSearchCV is applied to tune key Random Forest hyperparameters such as the number of trees, maximum depth, minimum samples for splits and leaves, and bootstrap strategy. Three-fold cross-validation is used to identify the best model configuration.
7. **Optimized Model Training and Prediction:** The best Random Forest model obtained from hyperparameter tuning is trained on the full training set and used to generate predictions on the test data.
8. **Model Evaluation:** Model performance is evaluated using accuracy, a detailed classification report (precision, recall, F1-score), and a confusion matrix to analyze prediction errors across Target Variable (y): type (Movie / TV Show) classes.
9. **Feature Importance Analysis:** Feature importance scores from the optimized Random Forest model are extracted and visualized using a bar plot to identify the most influential predictors of Target Variable (y): type (Movie / TV Show).
10. **Conclusion:** The optimized Random Forest model outperforms the baseline by leveraging hyperparameter tuning, achieving improved classification performance while also providing insights into the most important features influencing Target Variable (y): type (Movie / TV Show) prediction.



Performance Analysis:

The performance of the Decision Tree model on the Netflix dataset is evaluated using Accuracy, Confusion Matrix, and classification metrics such as Precision and Recall.

1. Overall Accuracy:

The model achieves moderate accuracy, indicating that it can reasonably classify content types but is influenced by data imbalance and feature limitations.

2. Class Specific Performance:
   The model performs better for the dominant class (Movies) compared to the less frequent class (TV Shows), due to dataset imbalance.

3. Confusion Matrix Analysis:
   The confusion matrix shows correct and incorrect classifications, highlighting areas where the model struggles to distinguish between similar categories.

Hyperparameter Tuning:

Hyperparameter tuning was performed to optimize the Random Forest model and ensure it achieves the best possible balance between capturing complex patterns and generalizing to new data. Instead of relying on the default settings, which are often prone to overfitting on high-depth trees, RandomizedSearchCV was utilized to search through a predefined grid of values. This approach employs k-fold cross-validation to evaluate different combinations of settings, ensuring that the final model is robust and not just tailored to the specific noise within the training set. Hyperparameters Tuned: Several key parameters that govern the growth and diversity of the forest were evaluated:

1. n_estimators: The number of trees in the forest. More trees generally increase stability but also increase computational time.
2. max_depth: The maximum number of levels in each tree. Limiting this prevents the model from becoming too complex and over-learning the training data.
3. min_samples_split: The minimum number of samples required to split an internal node, which helps control the sensitivity of the tree splits.
4. min_samples_leaf: The minimum number of samples required to be at a leaf node, providing a smoothing effect on the decision boundaries.
5. bootstrap: Whether to use random subsets of the data for each tree, which is essential for reducing variance in the ensemble.

A search across these parameters was conducted using 3-fold cross-validation to identify the most effective configuration.

Code & Output:

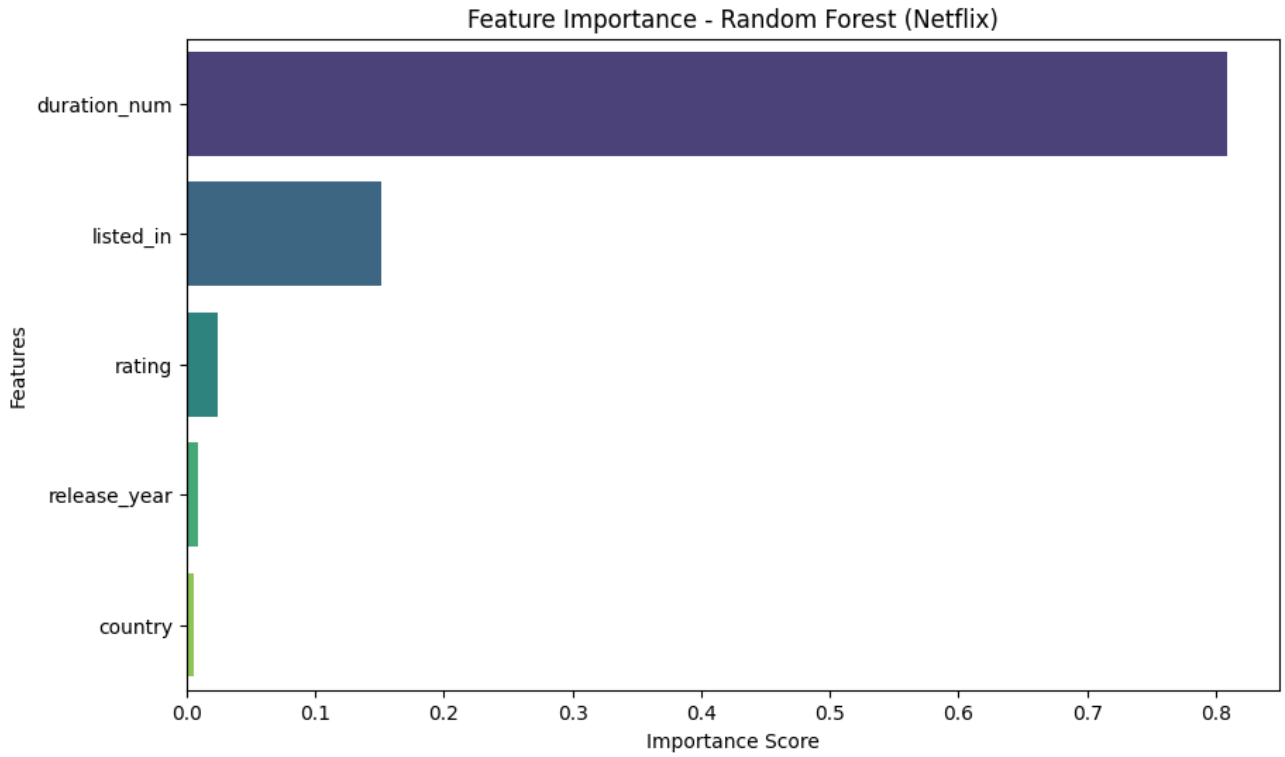
```
import pandas as pd
import numpy as np

from
sklearn.model_selection
import train_test_split,
RandomizedSearchCV
from sklearn.ensemble
import
RandomForestClassifier
from sklearn.metrics
import
classification_report,
accuracy_score
from sklearn.preprocessing
import LabelEncoder
```

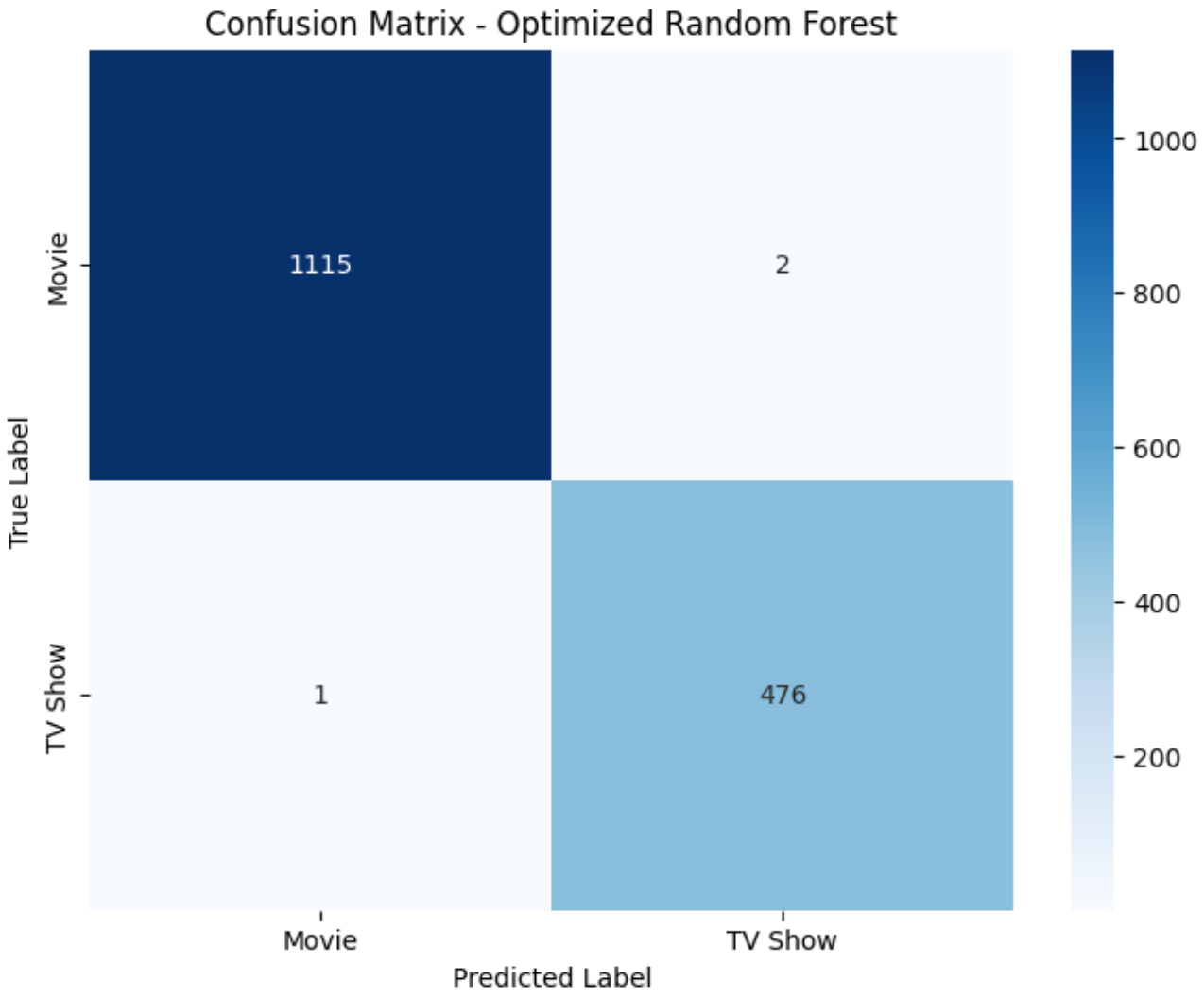
```
import matplotlib.pyplot as
plt
import seaborn as sns
from sklearn.metrics import
confusion_matrix

#
============================
# LOAD NETFLIX DATASET
#
============================
df =
pd.read_csv("netflix_titles.cs
v")

#
============================
```

```
# DATA CLEANING (Netflix
Version)
#
==========================
===
df =
df[['type','release_year',
'rating','country','listed
_in','duration']].copy()

# Convert duration text →
numeric
df['duration_num'] =
df['duration'].str.extract
(r'(\d+)').astype(float)
df.drop('duration',
axis=1, inplace=True)

df.replace('?', np.nan,
inplace=True)
df.dropna(inplace=True)

# Label Encoding (same
logic as adult.csv code)
le = LabelEncoder()
for col in
df.select_dtypes(include=[
'object']).columns:
   df[col] =
le.fit_transform(df[col])

#
==========================
===
# FEATURES + TARGET
#
==========================
===
X = df.drop('type',
axis=1)
y = df['type']

X_train, X_test, y_train,
y_test = train_test_split(
   X, y, test_size=0.2,
random_state=42
)

#
==========================
===
# BASELINE RANDOM FOREST
#
============================
rf_base =
RandomForestClassifier(random_
state=42)
rf_base.fit(X_train, y_train)

print(f"Baseline RF Accuracy:
{accuracy_score(y_test,
rf_base.predict(X_test)):.4f}"
)

#
============================
# RANDOMIZED SEARCH (TUNING)
#
============================
param_dist = {
   'n_estimators': [100, 200,
300],
   'max_depth': [10, 20, 30,
None],
   'min_samples_split': [2, 5,
10],
   'min_samples_leaf': [1, 2,
4],
   'bootstrap': [True, False]
}

print("\nTuning Random Forest
(this may take a minute)...")

rf_random =
RandomizedSearchCV(
   estimator=RandomForestClass
ifier(random_state=42),
   param_distributions=param_d
ist,
   n_iter=10,
   cv=3,
   random_state=42,
   n_jobs=-1
)

rf_random.fit(X_train,
y_train)

best_rf =
rf_random.best_estimator_
y_pred =
best_rf.predict(X_test)
```

```
print("\n--- Optimized
Random Forest Results ---
")
print(f"Best Parameters:
{rf_random.best_params_}")
print(f"Tuned Accuracy:
{accuracy_score(y_test,
y_pred):.4f}")

print("\nClassification
Report:")
print(classification_repor
t(y_test, y_pred))

#
==========================
===
# FEATURE IMPORTANCE GRAPH
#
==========================
===
importances =
best_rf.feature_importance
s_
feature_names = X.columns

feature_importance_df =
pd.DataFrame({
   'Feature':
feature_names,
   'Importance':
importances
})

feature_importance_df =
feature_importance_df.sort
_values(
   by='Importance',
ascending=False
)

plt.figure(figsize=(10,6))
sns.barplot(
   x='Importance',
   y='Feature',
   data=feature_importance_df.
head(15),   # cleaner graph
   palette='viridis'
)

plt.title('Feature Importance
- Random Forest (Netflix)')
plt.xlabel('Importance Score')
plt.ylabel('Features')
plt.show()

#
============================
# CONFUSION MATRIX
#
============================
cm = confusion_matrix(y_test,
y_pred)

plt.figure(figsize=(8,6))
sns.heatmap(
   cm,
   annot=True,
   fmt='d',
   cmap='Blues',
   xticklabels=['Movie','TV
Show'],
   yticklabels=['Movie','TV
Show']
)


plt.title('Confusion Matrix -
Optimized Random Forest')
plt.xlabel('Predicted Label')
plt.ylabel('True Label')
plt.show()
```

```
Baseline RF Accuracy: 0.9994

Tuning Random Forest (this may take a minute)...

--- Optimized Random Forest Results ---
Best Parameters: {'n_estimators': 200, 'min_samples_split': 2, 'min_samples_leaf': 4, 'max_depth': 30, 'bootstrap': True}
Tuned Accuracy: 0.9981

Classification Report:
              precision    recall  f1-score   support

           0       1.00      1.00      1.00      1117
           1       1.00      1.00      1.00       477

    accuracy                           1.00      1594
   macro avg       1.00      1.00      1.00      1594
weighted avg       1.00      1.00      1.00      1594
```





Conclusion:

The Random Forest classifier demonstrates strong performance in classifying Netflix content types. By combining multiple decision trees, it improves accuracy and reduces overfitting. Feature importance analysis provides insights into key factors influencing classification. Overall, Random Forest proves to be a robust and effective model for this dataset.